

MIN / MAX ALGORITHM

AIM:

To determine the optimal move in a two-player game by simulating all possible moves, assuming both players play optimally.

```
1  import math
2  def check_win(board, player):
3      for i in range(3):
4          if all(board[i][j] == player for j in range(3)):
5              return True
6          if all(board[j][i] == player for j in range(3)):
7              return True
8      if board[0][0] == player and board[1][1] == player and board[2][2] == player:
9          return True
10     if board[0][2] == player and board[1][1] == player and board[2][0] == player:
11         return True
12     return False
13 def is_draw(board):
14     return all(board[i][j] != 0 for i in range(3) for j in range(3))
15 def minmax(board, depth, is_maximizing):
16     if check_win(board, 1):
17         return 1
18     if check_win(board, -1):
19         return -1
20     if is_draw(board): # Draw
21         return 0
22     if is_maximizing:
23         best = -math.inf
24         for i in range(3):
25             for j in range(3):
26                 if board[i][j] == 0:
27                     board[i][j] = 1
28                     best = max(best, minmax(board, depth + 1, False))
29                     board[i][j] = 0
30         return best
31     else:
32         best = math.inf
33         for i in range(3):
34             for j in range(3):
35                 if board[i][j] == 0:
```

```

35         if board[i][j] == 0:
36             board[i][j] = -1
37             best = min(best, minmax(board, depth + 1, True))
38             board[i][j] = 0
39     return best
40 def best_move(board):
41     best_val = -math.inf
42     move = (-1, -1)
43     for i in range(3):
44         for j in range(3):
45             if board[i][j] == 0:
46                 board[i][j] = 1
47                 move_val = minmax(board, 0, False)
48                 board[i][j] = 0
49                 if move_val > best_val:
50                     move = (i, j)
51                 best_val = move_val
52     return move
53 def print_board(board):
54     for row in board:
55         print(" ".join(str(cell) for cell in row))
56     print()
57 if __name__ == "__main__":
58     board = [
59         [1, -1, 1],
60         [0, -1, 0],
61         [0, 1, 0]
62     ]
63     print("Current Board:")
64     print_board(board)
65     move = best_move(board)
66     print(f"Best move for X: {move}")

```

RESULT:

The best move for the current player that maximizes their minimum guaranteed outcome (for the maximizer) or minimizes the opponent's maximum gain (for the minimizer).